Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

К защите допустить:
Заведующий кафедрой
_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к расчетной работе
по дисциплине «Проектирование программного обеспечения интеллектуальных систем»:

Найти дополнение неориентированного графа

Студент: И. А. Политыко
Группа: 321701

Руководитель: М. Е. Садовский

Минск 2024

ВВЕДЕНИЕ

В современном мире графовые структуры играют ключевую роль в моделировании различных объектов и систем, от социальных сетей и транспортных маршрутов до биологических и коммуникационных сетей. Одной из важных задач, связанной с графами, является нахождение пересечения двух графов, то есть построение нового графа, в котором сохраняются только те связи, которые присутствуют в обоих исходных графах. Данная задача позволяет выделять ключевые общие элементы в сложных структурах, что находит применение в анализе социальных сетей, биоинформатике и логистике. Полученные результаты помогают улучшать точность анализа, выявлять пересечения данных и оптимизировать взаимодействие различных систем.

Цель работы — разработка агента для нахождения пересечения 2 заданных неориентированного графов. Агент должен автоматически определять общие вершины и связи между ними в двух заданных графов и формировать пересечение в виде нового графа, содержащего все общие вершины и ребра между вершинами.

Для достижения поставленной цели необходимо решить следующие задачи:

– Разработать алгоритм, который будет эффективно извлекать все вершины и вершины с которыми связана из исходных графов.

– Реализовать механизм для нахождения общих вершин и ребер между двумя графами.

– Организовать формирование структурированного результата для последующего использования в других задачах.

Графовые структуры и операции над ними являются важным инструментом для моделирования и анализа систем в таких областях, как информатика, социальные науки, логистика и биология. Нахождение пересечения графов позволяет выделять общие элементы сетевых структур, что важно для анализа взаимодействий, оптимизации процессов и выявления закономерностей в сложных системах. Автоматизация этой задачи с помощью специализированного агента сделает процесс анализа более быстрым и эффективным, сократит вероятность ошибок и облегчит работу с большими объемами данных. Это особенно важно при исследовании сложных сетей, где ручная обработка недостаточно точна и требует значительных затрат времени.

1 СПИСОК ИСПОЛЬЗОВАННЫХ ПОНЯТИЙ

а) ***Неориентированный граф***(абсолютное понятие) - граф, в котором все ребра являются звеньями, то есть порядок двух концов ребра графа не существенен

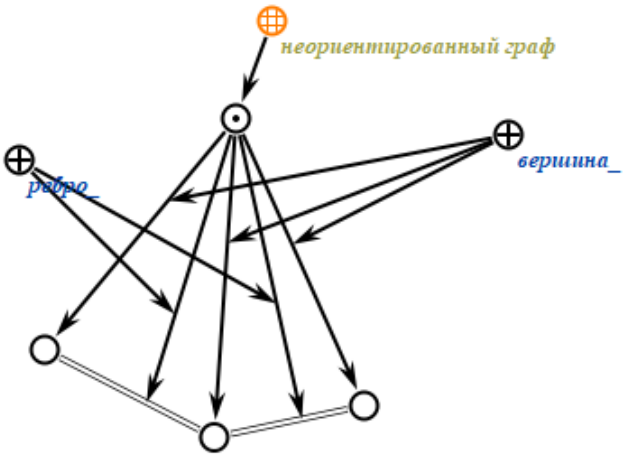

Рисунок *1.1* – Абсолютное понятие неориентированного графа

б) ***Пересечение в неориентированном графе***(абсолютное понятие) - это граф, в котором сохраняются только те вершины и ребра, которые одновременно присутствуют в обоих исходных графах. Таким образом, пересечение графов включает в себя общие элементы их структур, исключая все остальные связи и вершины.

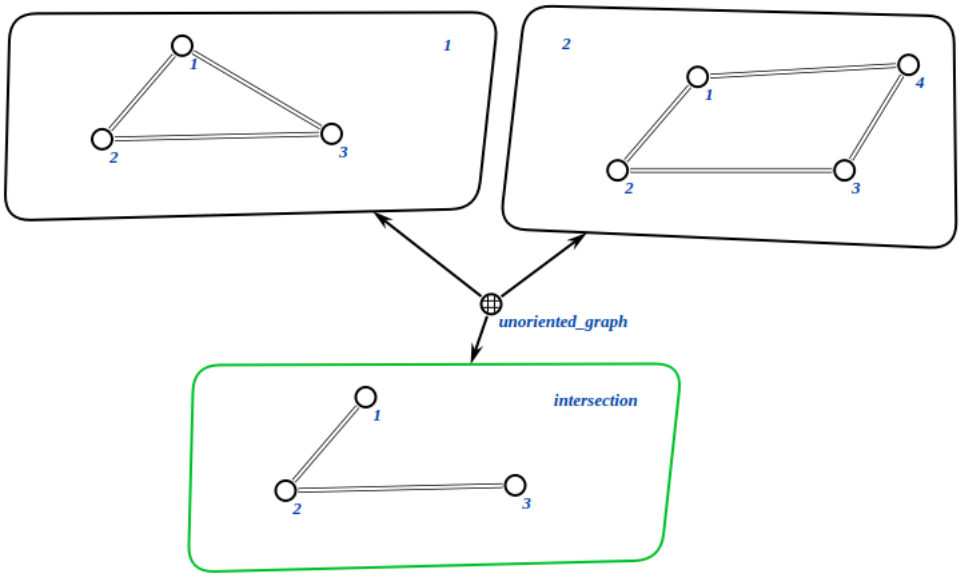

Рисунок *1.2* – Пример нахождения пересечения

2 АНАЛИТИКА

Для решения задачи нахождения пересечения двух графов был разработан агент, использующий цикличный алгоритм для поиска общих вершин и ребер между двумя графами. В основе работы алгоритма лежит последовательный обход всех вершин первого графа и проверка их наличия во втором графе. После того как общие вершины найдены, агент проверяет наличие общих ребер между этими вершинами в обоих графах и включает их в результирующий граф пересечения.

Идея алгоритма заключается в следующем: агент выполняет цикличный обход всех вершин первого графа и для каждой вершины ищет её присутствие во втором графе. Затем для каждой найденной общей вершины проверяется наличие соответствующего ребра между ней и другими вершинами. Если такие ребра есть и они присутствуют в обоих графах, то они добавляются в результат. Этот подход позволяет эффективно находить пересечение двух графов, сохраняя только общие элементы — вершины и ребра.

Преимущества данного метода включают его простоту и эффективность для графов средней плотности, так как алгоритм минимизирует количество проверок за счет цикличного обхода и фокусируется только на общих элементах. Использование такого подхода значительно ускоряет процесс нахождения пересечений по сравнению с более сложными методами, требующими полного перебора всех возможных связей.

Однако одним из ограничений является возможность увеличения времени обработки для очень крупных и плотных графов, так как алгоритм выполняет последовательные проверки для каждой вершины и ребра. Тем не менее, этот подход остается эффективным для большинства практических задач, где важно быстро получить пересечение двух графов и выделить общие элементы для дальнейшего анализа.

3 РАЗРАБОТКА

3.1 Глобальные переменные

Для работы алгоритма напрямую глобальные переменные не используются. Вся необходимая информация передается через параметры функций или инициализируется локально в пределах функций. Основные элементы, использующиеся в алгоритме, это:

– **mcontext** - объект, представляющий контекст, с помощью которого агент взаимодействует с памятью и выполняет операции с элементами графа.

– **versh** - структура для хранения вершины и всех других вершин с которыми она связана.

3.2 Алгоритм

Функция DoProgram - является основной функцией агента, которая реализует алгоритм создания пересечения неориентированных графов. Она принимает в качестве аргумента *ScAction action*, в котором содержится информация о графах, для которого необходимо построить пересечения.

Шаги функции DoProgram:

а) Получает аргументы *SetAddr1 и SetAddr2* из действия *action*, представляющий собой адрес графа, для которого нужно построить дополнение.

б) Проверяет, существует ли указанные элементы в памяти. Если хотя бы один граф не существует, агент завершает выполнение с ошибкой.

в) Создает пустой вектор *graph1* для хранения вершин типа*versh* первого графа.

г) Создает пустой вектор *graph2* для хранения вершин*versh* второго графа.

д) С помощью итератора *ScIterator3* перебирает все вершины первого графа по следующему циклу.

1) Создает переменную *temp*

2) Добавляет вершину графа в *temp.addres*

3) С помощью итератора *ScIterator5* перебирает все вершины графа связанные с вершинной из *temp.addres* и добавляет из в *temp.related.*

е) С помощью итератора *ScIterator3* перебирает все вершины второго графа по следующему циклу.

1) Создает переменную *temp*

2) Добавляет вершину графа в *temp.addres*

3) С помощью итератора *ScIterator5* перебирает все вершины графа связанные с вершинной из *temp.addres* и добавляет из в *temp.related.*

ж) Создает переменную *resultGraph*, в которой будет записан результат выполения функции *getIntersectionGraph*

з) Создает структуру *structure*

и) Начинает цикл по созданию графовой структуры перебирая каждую вершину из *resultGraph*

1) Записывает в структуру *structure* вершину

2) Начинает цикл по созданию ребер перебирая каждую вершину, с которой связана с записанной на предыдущем шаге выршины, из *resultGraph*

к) Записывает *structure* и завершает функцию

Функция getIntersectionGraph - вспомогательная функция, которая находит пересечения вершин и их ребер в между 2 графами. Она принимает на вход граф *graph1*, *graph2*, *resultGraph*.

Шаги функции getIntersectionGraph:

а) Начинает перебор вершин из *graph1* и *graph2*

б) Начинает перебор вершин из *graph1* и *graph2*

в) Находит общую вершину

1) Создает переменную *temp*

2) Создает пустой вектор *vershin* для хранения вершин

3) Начинает перебор вершин с которыми связана вершина в *graph1*

4) Начинает перебор вершин с которыми связана вершина в *graph2*

5) Найденную общую вершину записывает в *vershin*

6) Записывает *vershin* в *temp.related*

7) Записывает *temp* в *result*

г) Возвращает *result* и завершает функцию

Алгоритм для нахождения пересечения двух графов имеет сложность $O(V_1 \cdot V_2 + E_1 + E_2)$, где:

– V_1 и V_2 — количество вершин в первом и втором графах соответственно,

– E_1 и E_2 — количество рёбер в первом и втором графах соответственно.

4 ПОКАЗ ТЕСТОВЫХ СЦЕНАРИЕВ

а) **Тест 1:**

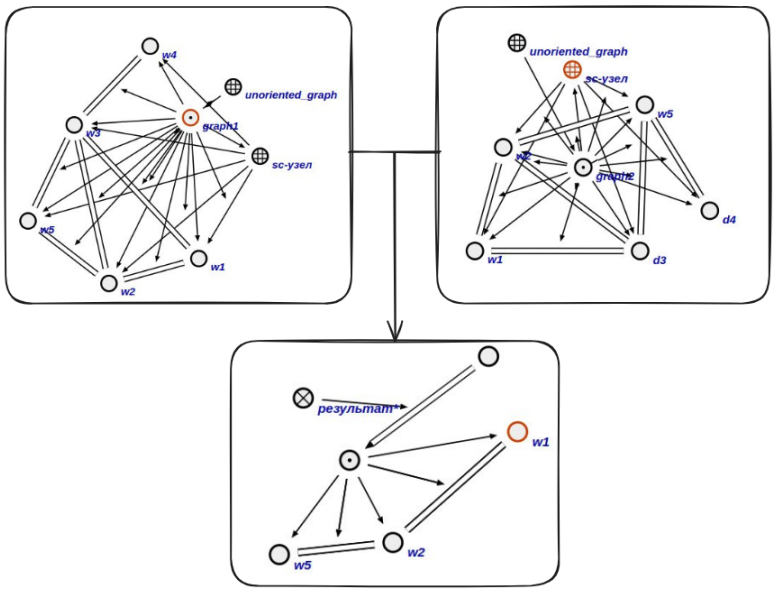

Рисунок *4.1* – Нахождение пересечения графов

б) **Тест 2:**

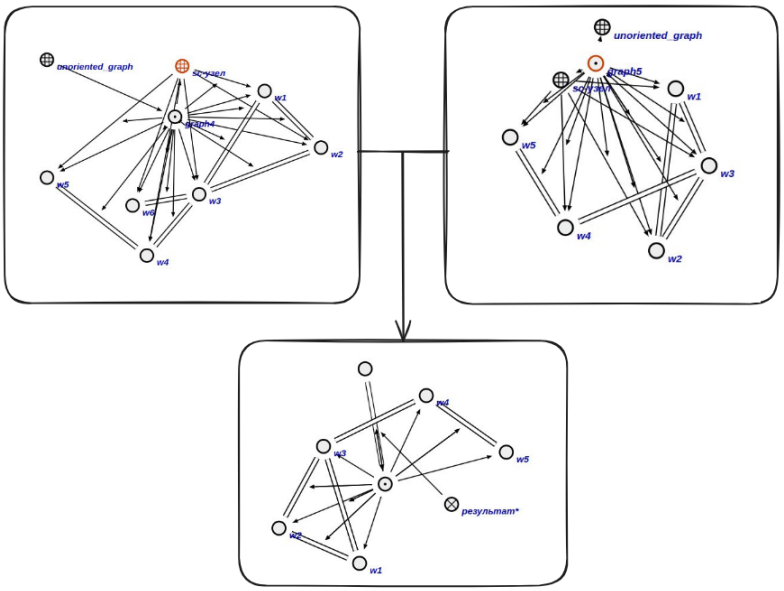

Рисунок *4.2* – Нахождение пересечения графов

в) **Тест 3:**

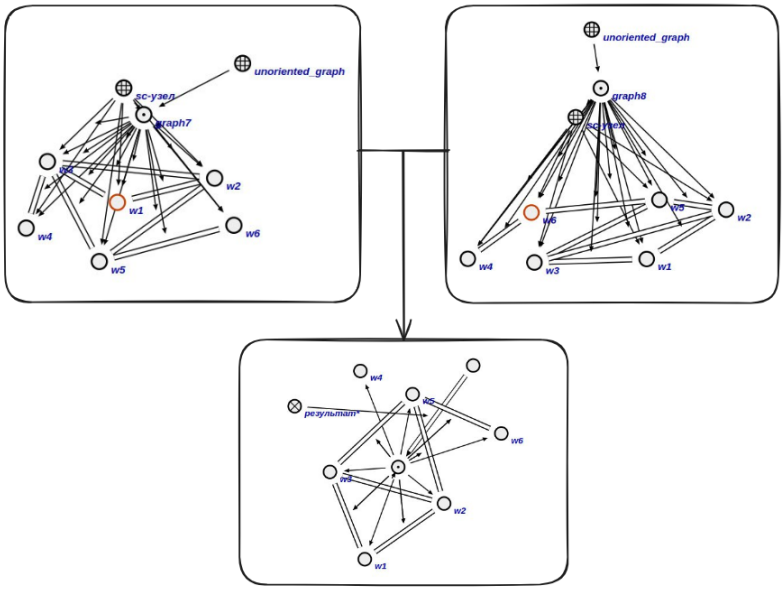

Рисунок *4.3* – Нахождение пересечения графов

г) **Тест 4:**

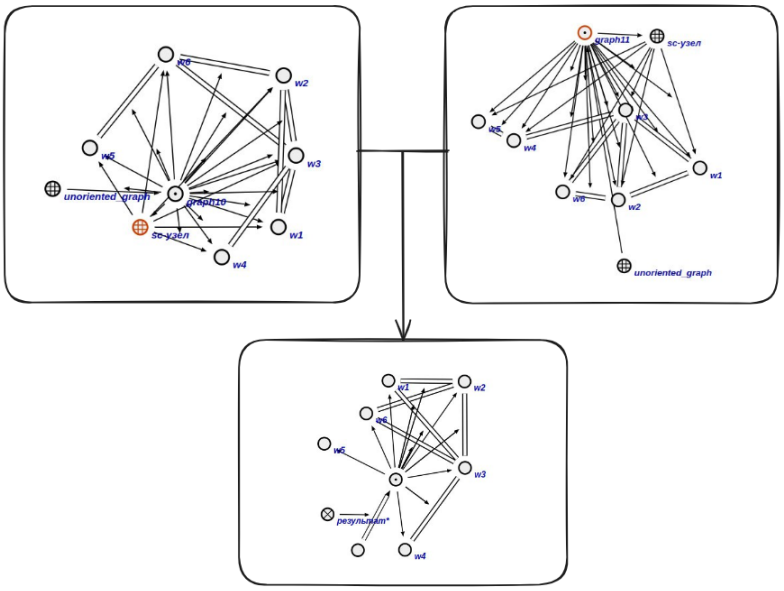

Рисунок *4.4* – Нахождение пересечения графов

д) **Тест 5:**

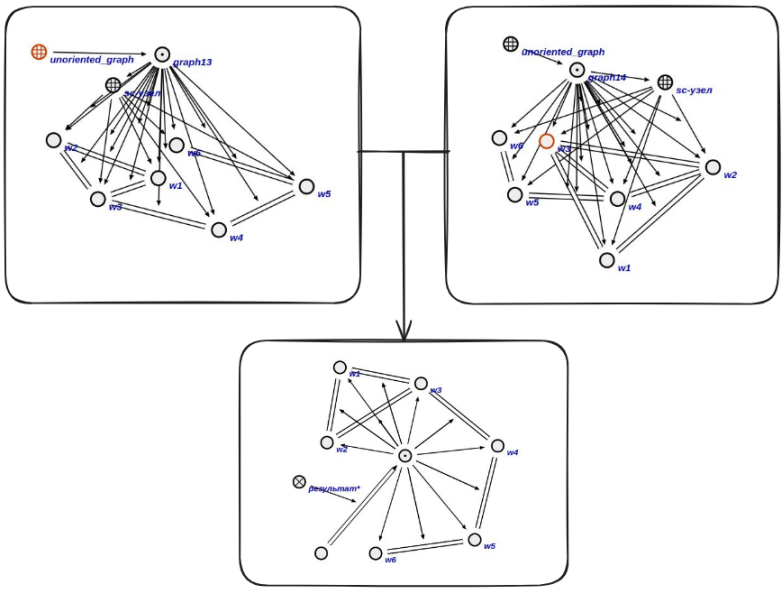

Рисунок *4.5* – Нахождение пересечения графов

ЗАКЛЮЧЕНИЕ

В ходе работы был разработан агент для автоматического нахождения пересечения двух заданных графов. Данный агент представляет собой инструмент, который позволяет анализировать общие элементы графовых структур, выделяя пересекающиеся вершины и ребра в результирующем графе. Работа агента основана на алгоритме, который выполняет цикличный обход первого графа, проверяет наличие общих вершин и соответствующих связей во втором графе, формируя итоговую пересеченную структуру.

Агент успешно решает поставленную задачу благодаря своей продуманной структуре и использованию эффективных методов работы с графами, таких как итеративный обход вершин и проверка ребер на совпадение. Создание и использование данного агента имеет практическую значимость для решения множества задач. Пересечение графов может быть полезно в анализе социальных сетей для выявления общих связей между группами пользователей, в биоинформатике для исследования совпадающих взаимодействий генов или белков, а также в логистике для оптимизации совпадающих маршрутов различных операторов.

Таким образом, *ScAgentGraphIntersection* — это не только инструмент для решения конкретной задачи, но и шаг к созданию более универсальных методов анализа сложных графовых структур.

Подводя итоги, можно отметить, что работа достигла поставленных целей. Агент был успешно разработан и протестирован, что подтверждает его работоспособность и эффективность. Полученные результаты открывают новые возможности для применения агента в прикладных и научных задачах, где анализ графов и их пересечений позволяет глубже понять структуру, свойства и взаимодействия систем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Харари, Фрэнк. Теория графов / Фрэнк Харари. — 1973.

[2] Иванова, ГС. Полная характеристика структуры неориентированного графа / ГС Иванова, ВА Овчинников. No. 4. — Некоммерческое партнерство «Национальный Электронно-Информационный Консорциум», 2016. — Pp. 106–123.

[3] SC-machine [Электронный ресурс]. - Режим доступа: https://ostis-ai.github.io/sc-machine. - Дата доступа: 14.12.2024.